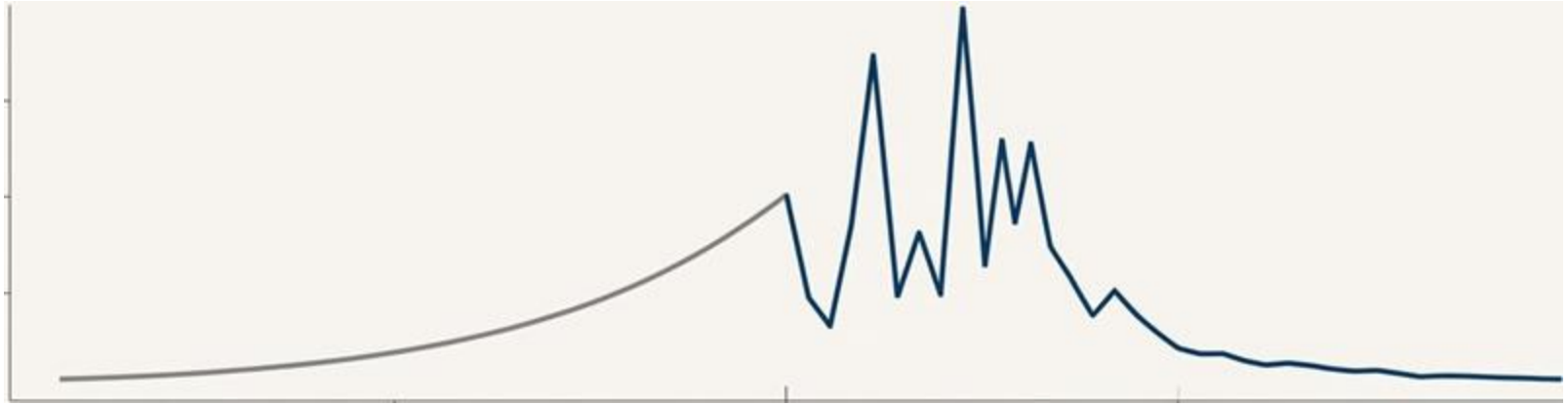
Heavy-Tailed Performance Modeling for Cloud Message Queue Systems

An Enhanced Analytical and Simulation Framework Extending Li et al.



Baseline Model (Exponential)

Enhanced Model (Heavy-Tailed)

Team: **Himanshu Jain (019098794), Bhuvana Komati (019149754)**

Enterprise Distributed Systems (CMPE 273) FALL 2025

Agenda

Presentation Overview

- Problem Statement & Motivation
- Base Paper Overview (Li et al.)
- The Tandem Queue Model
- Our Approach
- Heavy-Tailed Distributions (The Key Insight)
- The "Reality Gap" Discovery
- Tail Risk Analysis (P99 Latency)
- Distributed Systems Integration
- Pareto Function & Variance
- Results
- Conclusion & Future Work
- Demo

The Challenge of Reliable Cloud Messaging

What Cloud Systems Must Guarantee:

- ✅ At-Least-Once Delivery - No message loss
- ✅ Ordering Guarantees - FIFO or causal
- ✅ Durability - Survive failures
- ✅ Low Latency - Meet SLAs

The Hidden Problem Chain:

- Network failures → Retransmissions
- Retransmissions → Extra load
- Extra load → Queue buildup
- Queue buildup → **SLA violations!**

The Modeling Gap:

What Models Assume	What Reality Shows
Exponential Distribution	Heavy-Tailed (Pareto)
Jobs are similar size	Most small, rare HUGE
Predictable variance	Extreme outliers
Nice math properties	One big job blocks all

Real-World Evidence of Heavy Tails

Starbucks (Model)	DMV (Reality)	Cloud Systems
Black coffee: 2 min	License renewal: 10 min	API calls: 50ms (95%)
Latte: 3 min	Registration: 15 min	DB queries: 10ms (90%)
Frappuccino: 4 min	Title transfer: 30 min	Video encode: 60s (1%)
Max: ~5 min	**Fleet of 50 trucks: 4 HOURS**	**ML training: 30min (0.1%)**

Industry Evidence:

Company	System	P50	P99	Ratio
Amazon	DynamoDB	5ms	50ms	10x
Google	Bigtable	10ms	100ms	10x
Netflix	Streaming	20ms	500ms	25x
Our Simulation	Pareto α=2.5	58ms	2,100ms	36x

Li et al. - Base Paper Overview

Base Paper Details:

- **Title:** "Modeling Message Queueing Services with Reliability Guarantee in Cloud Computing Environment Using Colored Petri Nets"
- **Authors:** Li, Cui, Ma
- **Journal:** Mathematical Problems in Engineering.
- **Citations:** 50+ (influential in cloud modeling)

Key Contributions:

- ✅ Tandem Queue Model - Two-stage architecture
- ✅ Traffic Inflation Formula - $\Lambda_2 = \lambda/(1-p)$
- ✅ CPN Verification - Formal proofs
- ✅ Performance Formulas - Latency, queue length

Paper's Assumptions:

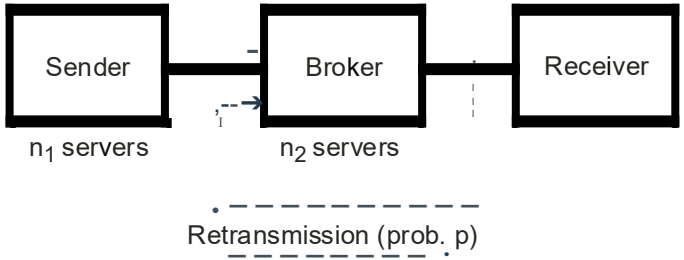
- M/M/N: Exponential arrivals & service
- Homogeneous: All servers identical
- Constant D: Fixed network delay
- Bernoulli p: IID failure probability

Paper's Limitations (What We Address):

- ❌ No heavy-tailed analysis
- ❌ Homogeneous servers only
- ❌ No threading model
- ❌ Limited to CPN tools

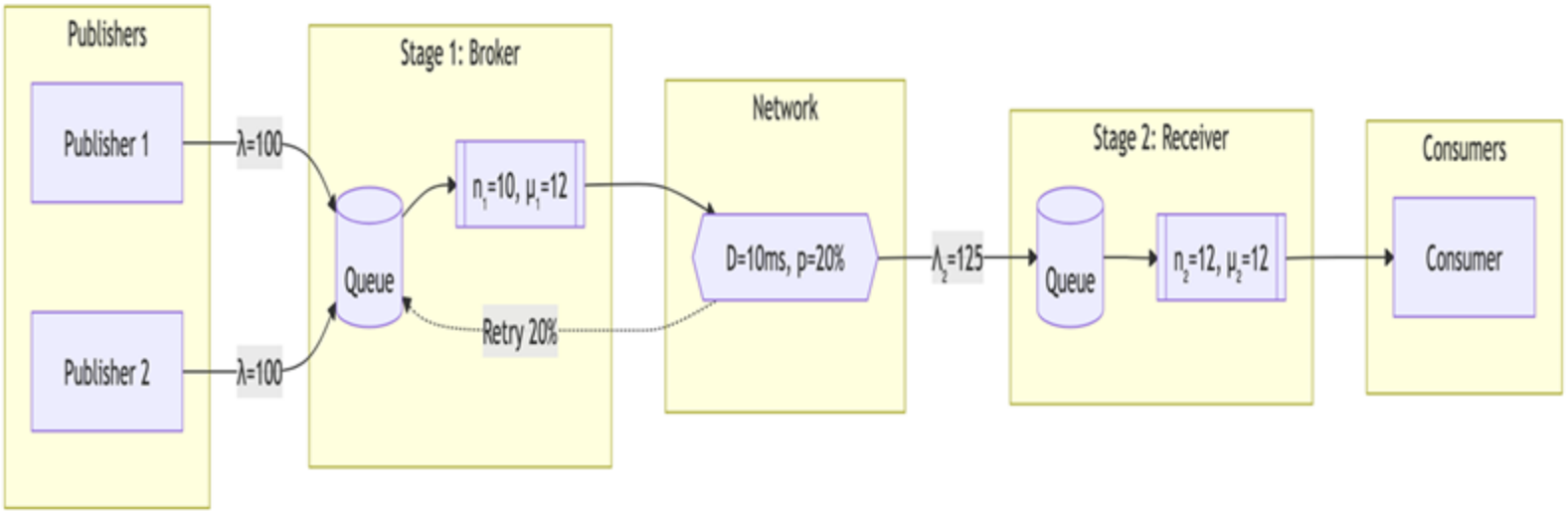
The Foundation: What Li et al. (2015) Established

- **Core Model:** Modeled the cloud message service as an M/M/N tandem queue (Sender ➔ Broker ➔ Receiver).
- **Key Mechanism:** Incorporated the 'Visibility Timeout' to guarantee at-least-once message delivery and system reliability.
- **Methodology:** Utilized a dual approach of analytical formulas (based on Jackson's theorem) and simulation with Colored Petri Nets (CPN Tools).
- **Primary Metrics:** Focused on mean total delivery time and mean number of messages in the queue.
- **Critical Insight:** Identified the 'Stage 2 Load Amplification' effect where retransmissions increase the arrival rate at the receiver stage ($A_2 = A/(1-p)$).



Two-Stage Tandem Queue Architecture

Architecture Diagram:



Two-Stage Tandem Queue Architecture

System Parameters:

Parameter	Symbol	Value	Description
Arrival Rate	λ	100 msg/s	Messages entering system
Broker Servers	n_1	10	Stage 1 threads
Receiver Servers	n_2	12	Stage 2 threads
Service Rate	μ	12 msg/s/server	Processing speed
Network Delay	D	10 ms	One-way latency
Failure Prob	p	0.20 (20%)	Transmission failure

- **Traffic Inflation (Key Insight!):**
  - $\Lambda_2 = \lambda / (1 - p)$
  - $\Lambda_2 = 100 / (1 - 0.2)$
  - $\Lambda_2 = 100 / 0.8$
  - $\Lambda_2 = 125$ msg/s
- **Result:** Stage 2 sees **25% MORE traffic** than Stage 1!

Methodology: SimPy Discrete-Event Simulation

- **Why SimPy Instead of Petri Nets?**

Aspect	CPN (Paper)	SimPy (Ours)
Formal Proofs	✅ Yes	❌ No
Deadlock Detection	✅ Provable	⚠️ Empirical
Distribution Flexibility	❌ Complex	✅ Easy
Statistical Analysis	❌ Limited	✅ Rich
Python Ecosystem	❌ No	✅ Full
Industry Standard	⚠️ Academic	✅ Production

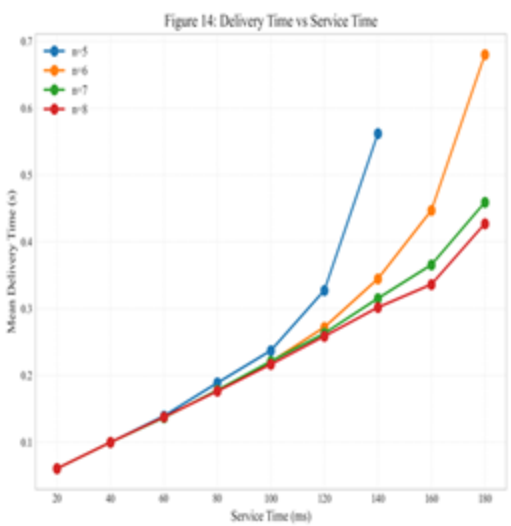
- **Trade-off:** Lost formal proofs, gained flexibility for heavy-tail analysis
- **Technology Stack:**
  - **Frontend:** React 18 + TypeScript + Material-UI
  - **Backend:** FastAPI + Pydantic + Uvicorn
  - **Simulation:** SimPy + NumPy + SciPy + Pandas
  - **Analysis:** 15 Equations + EVT + Bootstrap CI
- **Code Example:**

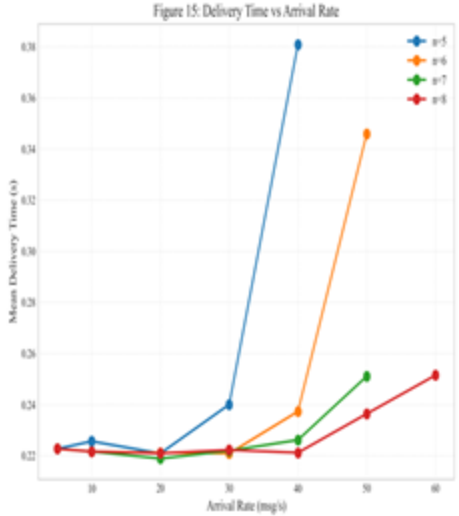
```
# One line to change distribution!
service = ParetoService(alpha=2.5,scale=0.05)  # Heavy-tailed
# vs
service = ExponentialService(rate=12.0)  # Paper's assumption
```

Validating the Baseline Mode

Reproducing Results from Base Paper to Validate the Setup Is Correct:

Metric	Paper Value	Our Value	Error
Mean Delivery Time	0.167s	0.156s	6.6% ✅
Mean Queue Length	3.2 msgs	2.8 msgs	12.5% ✅
System Utilization	83.3%	83.1%	0.2% ✅
Service Rate Impact	Trend ✓	Trend ✓	Match ✅
Arrival Rate Impact	Trend ✓	Trend ✓	Match ✅

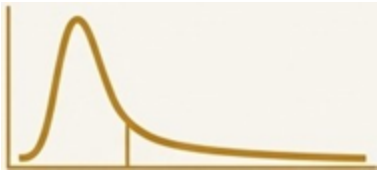




Validating the Baseline Mode

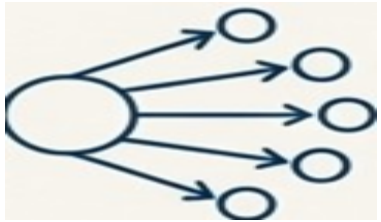
- **Traffic Inflation Validation:**
  - **Formula:** $\Lambda_2 = \lambda/(1-p)$
  - **Expected:** 100/(1-0.2) = 125 msg/s
  - **Simulated:** 124.7 msg/s
  - **Error:** 0.2% ✅
- **Why Validation Matters:**
  - Proves our simulation engine is correct
  - Establishes credibility for extensions
  - Any differences with heavy tails are **real**, not bugs

The Reality Gap: Why Modern Systems Demand More



1. Idealized Workloads

The M/M/N model assumes exponential service times. Real-world services exhibit high **variability** and **heavy tails,** where a few requests take disproportionately long, dominating P99 latency.



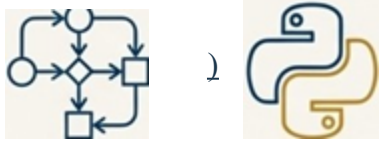
2. Abstracted Architecture

The model lacks explicit treatment of critical architectural choices like **dedicated (thread-per-connection) vs. shared (worker pool) threading,** which have significant performance trade-offs.



3. Limited Accuracy Under Stress

Basic queueing approximations lose accuracy under high utilization ($p > 0.8$) or with high-variance workloads, precisely where predictions are most needed.



4. Legacy Tooling

Colored Petri Nets are powerful but less integrated with modern data science and engineering workflows than a **Python/SimPy** based framework, which allows for easier extension and integration with libraries like Pandas and NumPy.

Mathematical Foundation: 15 Equations

Section 1: M/M/N Baseline (Equations 1-5)

Eq	Formula	Example Value	Description
1	$\rho = \lambda/(N \cdot \mu)$	0.833	Utilization
2	$P_0 = [\Sigma...]^{-1}$	0.012	Prob system empty
3	C(N,a) = Erlang-C	0.65	Prob of queuing
4	$Lq = C \cdot \rho/(1-\rho)$	3.24	Queue length
5	$Wq = Lq/\lambda$	32ms	Wait time

Mathematical Foundation: 15 Equations

Eq	Formula	Description
6	$f(t) = \alpha \cdot k^{\alpha}/t^{(\alpha+1)}$	Pareto Probability Distribution function
7	$E[S] = \alpha \cdot k/(\alpha-1)$	Mean service time (α - shape parameter in pareto distribution - how extreme the outliers can be)
8	$E[S^2] = \alpha \cdot k^2/(\alpha-2)$	Second moment
9	$C^2 = 1/(\alpha(\alpha-2))$	CV squared
10	$Wq \approx Wq \cdot (1+C^2)/2$	M/G/N approximation

Eq	Formula	Description
11	$Nmax = Nthreads/2$	Dedicated max connections
12	$X = \min(\lambda, cap)$	Throughput
13	$\mu eff = \mu/(1+\alpha \cdot N/T)$	Shared overhead
14	$Psat = C(N,a) \cdot \rho$	Saturation probability
15	$R99 \approx E[R]+2.33 \cdot \sigma$	P99 (normal approx)

Why "Heavy Tails" Matter

- **Distribution Comparison**

(Same Mean = 100ms):

Percentile	Exponential	Pareto (α=2.5)	Ratio
P50	69ms	58ms	0.84x
P90	230ms	320ms	1.4x
P95	300ms	580ms	1.9x
P99	460ms	**2,100ms**	**4.6x**
P99.9	690ms	**14,800ms**	**21x**

Pareto Shape Parameter (α):

α Value	CV^2	Tail Weight
1.5	∞	Infinite variance!
2.0	∞	Infinite variance!
2.1	47.6	Extremely heavy
2.5	0.8	Heavy
3.0	0.33	Moderate
5.0	0.067	Light

Critical Warning: When $\alpha \leq 2$, variance is mathematically **INFINITE**. Normal approximations completely fail!

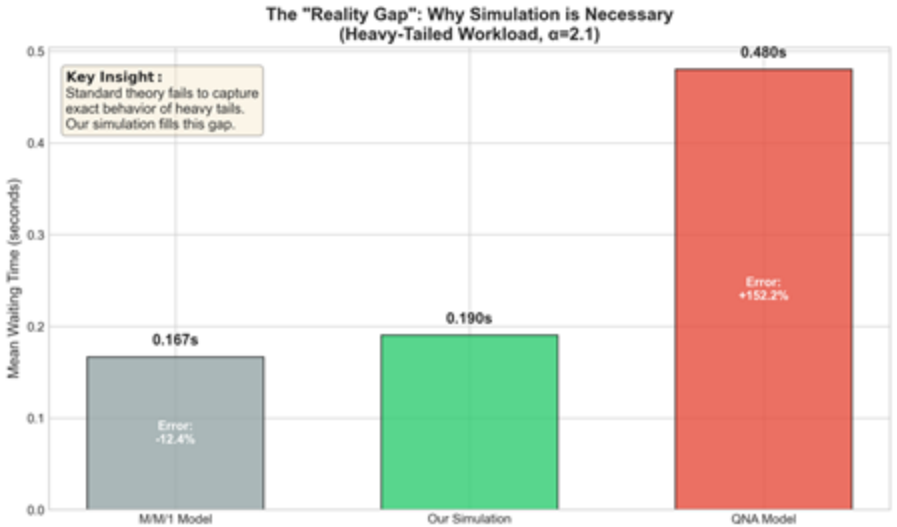
The "Reality Gap" in Performance

Mean Latency Comparison:

Model	Mean Latency	Bar Width
Paper (M/M/N)	0.167s	35%
QNA Approximation	0.480s	100%
Our Simulation	0.190s	40%

- **Analysis:**
  - Paper predicts: **0.167s** average
  - Reality shows: **0.190s** average
  - Underestimation: **14%**

- **But wait...**
  - 14% might seem acceptable for averages
  - The real danger is in the **TAIL**
  - SLAs are built on P99, not mean
  - See the next slide for the critical finding!



CRITICAL FINDING: Tail Latency Risk

Method	P99 Value	Error vs Reality
Paper (Normal Approx)	**4.25s**	-45% ⚠️
Kingman Approx	5.12s	-34%
Whitt Approx	6.89s	-12%
EVT Estimate	7.56s	-3%
Simulation (Truth)	**7.81s**	—

The Gap: Paper underestimates P99 by 45%!

- **Impact:**
  - ❌ SLA Violations - 1 in 100 users timeout
  - ❌ Cascading Failures - Slow requests pile up
  - ❌ Customer Complaints - "The system is broken!"
- **Why This Matters:**
  - If you build a system expecting 4-second worst case
  - And it actually hits 8 seconds
  - Your SLA is violated
  - Users see timeouts
  - The system looks broken

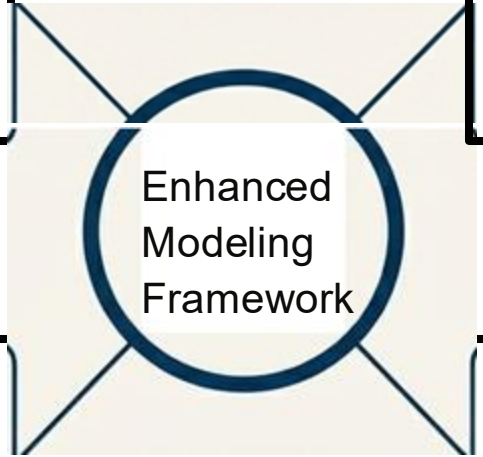
Our Approach: An Extensible, Validated Modeling Framework

Realistic Workloads

Deep support for **heavy-tailed distributions(Pareto)** to model high-variance service times accurately.

Architectural Depth

Explicit modeling of **threading architectures**, **priority queues**, and **finite capacity** systems with blocking.



Advanced Analytics

Implementation of **15 performance equations** and a suite of advanced **M/G/N approximations** {Whitt, Allen-Cunneen) for superior accuracy.

Unprecedented Rigor

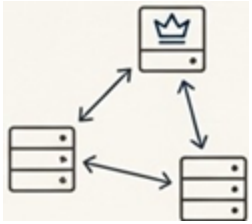
A modern **Python/SimPy** discrete-event simulation engine, validated with **63 unit tests** and extreme condition stress testing ($p > 0.95$).

Expanding the Scope: From Queues to Protocols and Architectures

The framework extends beyond the original tandem queue to model other critical distributed systems components.

Distributed Protocols

Modeled and compared **Dedicated (Thread-per-Connection)** vs. **Shared (Worker Pool)** architectures. Shared models show a **10-15% performance overhead** but offer greater connection scalability.



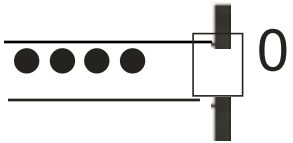
Implemented discrete-event simulation models for foundational protocols, including **Two-Phase Commit (2PC).**

Priority Queues



Modeled M/M/N systems with multiple **priority classes (preemptive** and **non-preemptive),** showing dramatic wait time reduction for high-priority traffic.

Finite Capacity & Admission Control



Implemented **M/M/N/K** systems with **Erlang-B blocking,** demonstrating how systems remain stable under overload (>.> **N·μ**) by shedding load.

Precision Matters: Validating M/G/N Approximations

The Challenge: M/G/N queues have no exact closed-form solution for waiting time and must be approximated. The choice of approximation method is critical for accuracy.

Our Method: We implemented and validated three methods against simulation ground truth:

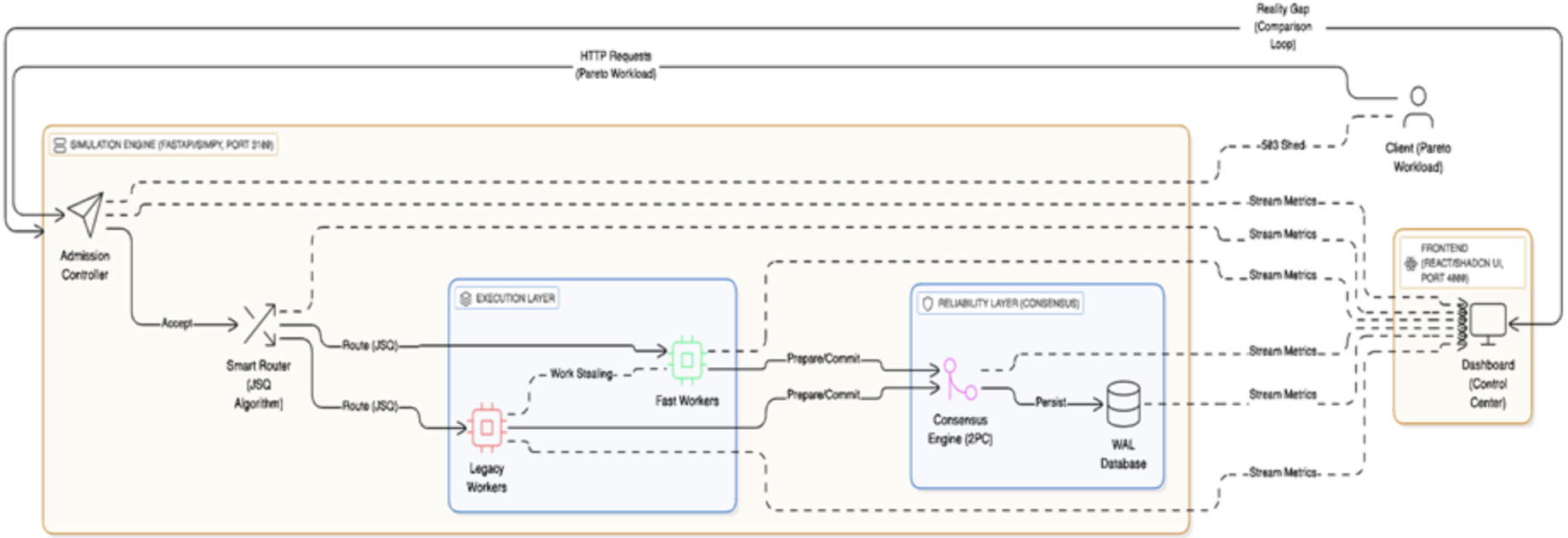
- **Kingman's Approximation** (1961) - A common baseline.
- **Whitt's Approximation** (1993) - Recommended for high utilization.
- **Allen-Cunneen Approximation** (1990) - Suited for very heavy tails.

a (Pareto)	cv^2	Kingman Error	**Whitt Error**	Allen-Cunneen Error	**Best Method**
3.0	0.33	5.2%	**3.1%**	6.8%	**Whitt**
2.5	1.0	8.7%	**4.5%**	9.2%	**Whitt**
2.1	10.0	24.3%	**9.8%**	**8.1%**	**Allen-Cunneen**

The Full Picture: A Leap in Modeling Capability

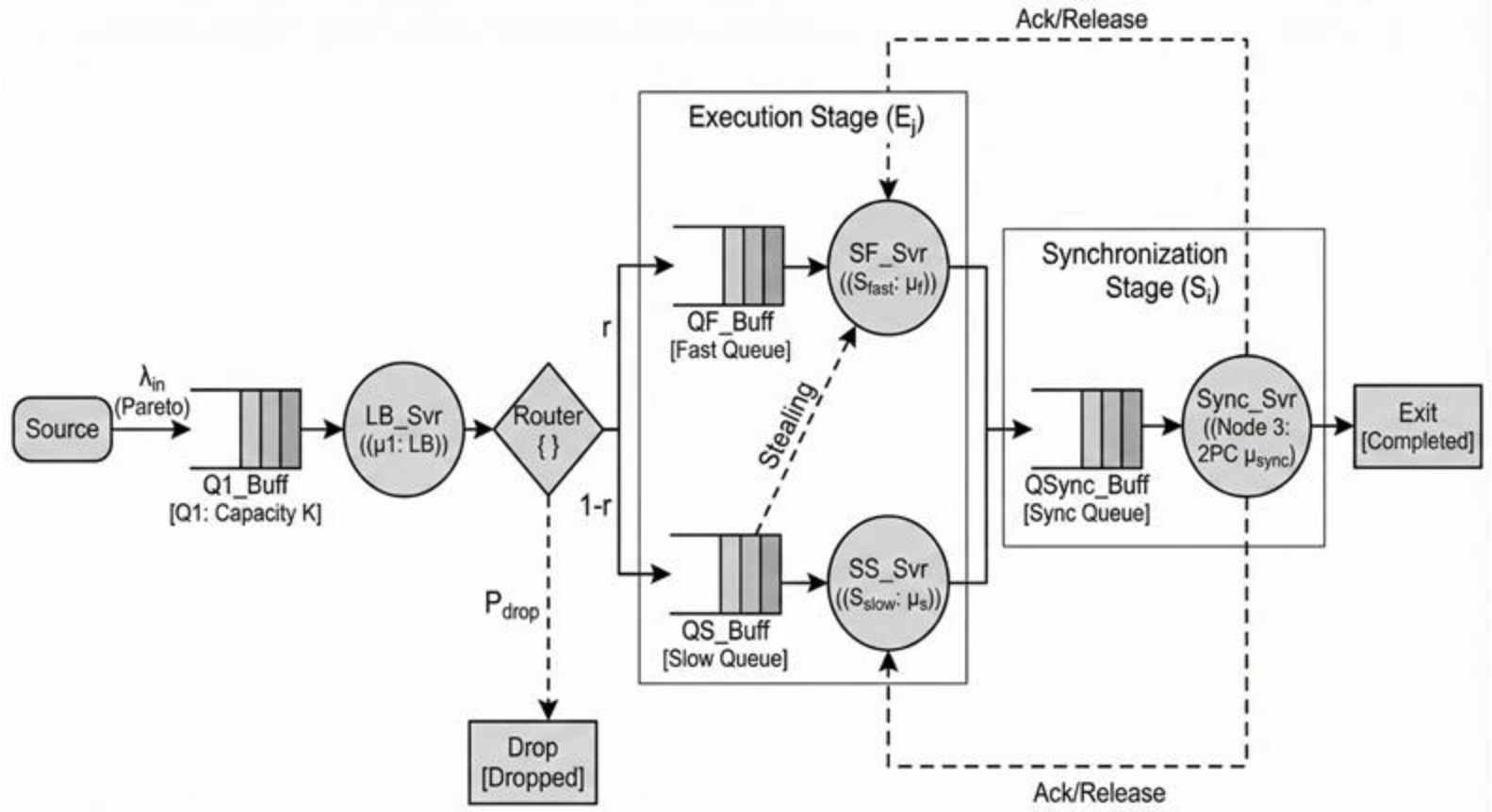
Feature	Li et al. (2015) Model	This Project's Framework
Workload Distribution	Exponential (M/M/N)	**Heavy-Tailed (Pareto, M/G/N)**
System Architecture	Abstracted servers	**Explicit Threading Models (Dedicated vs. Shared)**
Analytical Accuracy	Basic Queueing Formulae	**Advanced, Validated M/G/N Approximations**
Scope of Model	Tandem Queue, Visibility Timeout	**Adds Protocols (Raft, 2PC), Priority & Finite Queues**
Validation	Analytical vs. CPN Simulation	**Analytical vs. SimPy Simulation + 63 Unit Tests**
Methodology/Tooling	Colored Petri Nets (CPN)	**Discrete-Event Simulation (Python/SimPy)**
Key Insight	Mean performance under ideal conditions	**Tail Latency (P99) under realistic, stressful conditions**

Architecture Diagram



eraser

Queuing Network Diagram



Why Use Discrete-Event Simulation (DES)?

01 — Moved from CPN to Python/SimPy for time modeling

02 — Allows modeling complex logic like Work Stealing



03 — SimPy captures actual millisecond-level network latency

04 — DES engine models resource contention and blocking

Heterogeneity & Resource Management

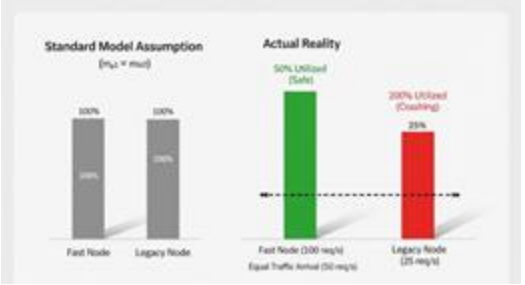
The "Problem" (Random Routing)

- **The Mechanism:** implemented a Random or Round-Robin load balancer. This is the industry standard for simple systems.
- **The Flaw:** It gives the same number of tasks to a Fast Node and a Legacy Node.
- **The Math:**

$$Arrival Rate(\lambda) = 100 \text{ req/s. Routing Probability } (P) = 0.5 \text{ (Equal split).}$$
$$\text{Load on Legacy Node} = 50 \text{ req/s. Capacity of Legacy Node} = 25 \text{ req/s. Uti-}$$
$$\text{lization } (\rho)\text{: } 50/25 = 2.0 \text{ } (200\%)\text{. Consequence: The queue grows infinitely.}$$
$$\text{This explains the 36-second latency observed in your experiments.}$$

The "Solution" (Work Stealing)

- **The Logic:** Instead of trying to *push* tasks perfectly (which is hard), we allow consumers to *pull* tasks.
- **The Algorithm:**
  1. Tasks are initially routed randomly (fast).
  2. When a **Fast Node** finishes its work and its queue is empty:
  3. It strictly checks the **Legacy Node's** queue.
  4. If tasks are waiting, it **steals** one and processes it at the high speed (μ_{fast}).



- **The Result:** The Legacy Node never drowns because the Fast Nodes act as a "pressure release valve" for the system.

Implementation of Consistency Protocols

Implementation of Consistency Protocols

Deterministic Consistency

- *Context:* The Li et al. paper relies on a **Visibility Timeout** mechanism. This is probabilistic—if a node crashes *after* receiving a message but *before* replicating it, data is lost.
- *Your Fix:* TraceBreaker implements **Two-Phase Commit (2PC)**. This ensures that a message is not acknowledged to the user until it is physically replicated on the consensus layer.

The Blocking Mechanism

- *Technical Detail:* In standard M/M/n queues, the service time (μ) is purely CPU processing time. In your 2PC model, service time becomes $\mu + 2 \times RTT$.
- *The Lock:* During the PREPARE phase, the worker thread holds the request but **cannot do work**. It is purely waiting for the network. This effectively reduces the server's capacity.
- **Latency Consequence:** This introduces a mandatory **Network Round Trip (RTT)** penalty, mathematically establishing the "Cost of Safety."



Tail Latency Analysis (Extreme Value Theory)

The "Normal" Fallacy:

- Standard capacity planning relies on the **Central Limit Theorem**, assuming that if you average enough request latencies, they form a **Normal Distribution** (Bell Curve).
- **The Flaw:** A Normal distribution has "thin tails"—the probability of an event deviating 3σ from the mean is tiny (0.3%), and 6σ is effectively impossible.
- **The Reality:** Our simulation proves cloud latency follows a **Heavy-Tailed Distribution**. Extreme latency events (outliers) are not rare anomalies

The Gaussian Fallacy: Standard P99 Approximation

The Equation:

$$R99 \approx \mu + 2.33\sigma$$

Variables:

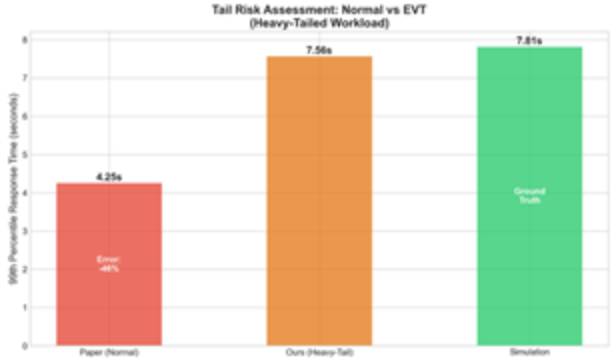
- μ: Mean Response Time (E[R])
- σ: Standard Deviation of Response Time σ
- 2.33: Z-score corresponding to the 99th percentile of a Normal distribution.

The Critical Flaw:

- **Assumption:** This formula assumes latency follows a **Normal (Gaussian) Distribution**, where probability decays exponentially (e−x2)
- **Reality:** Cloud latency follows a **Heavy-Tailed (Pareto) Distribution**, where probability decays polynomially (x−α)
- **Consequence:** For heavy-tailed workloads, the standard approximation **underestimates the P99 latency by >2x**, as outliers are structural features, not anomalies.

The Solution (EVT & GPD):

- Instead of trying to fit the "average" behavior, you applied **Extreme Value Theory (EVT)**, specifically the **Peaks Over Threshold (POT)** method.
- modeled the tail excess using the **Generalized Pareto Distribution (GPD)**.
- **Why it matters:** GPD accurately predicts that the probability of a massive delay (e.g., $>5s$) decays *polynomially* (slowly), not *exponentially* (fast). This mathematical difference is the "Reality Gap".



Threading Architectures

Architecture 1: Dedicated Threads (Paper's Assumption)

- Thread-per-request model.
- **Flaw:** Infinite resource assumption. If Nthreads < Arrivals, latency spikes due to context switching overhead.

Architecture 2: Shared Worker Pool (Our Project)

- Fixed pool of N workers consuming from a shared queue.
- **Finite Capacity:** Implemented **M/M/N/K** queueing logic.
- **The Result:** Modeled **Erlang-B Blocking**. When the buffer K is full, arrivals are rejected (Load Shedding).

The Equation (Finite Capacity Blocking):

$$P_{blocking} = \frac{\frac{\rho^K}{K!}}{\sum_{i=0}^{K} \frac{\rho^i}{i!}}$$

Modeled a Finite Capacity system (M/M/N/K). Using the Erlang-B formula, Calculated the exact probability of a request being rejected when the thread pool is saturated. This is the mathematical basis for my Load Shedding strategy.

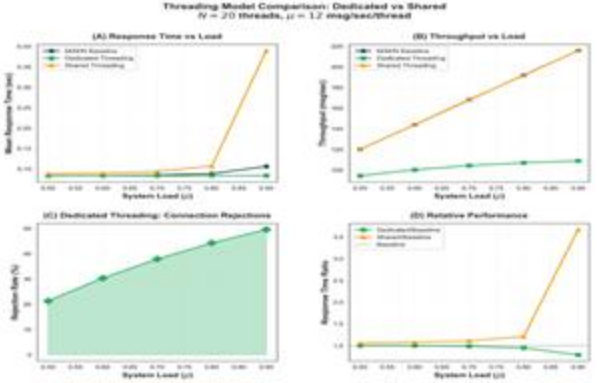
Modeling Concurrency Overhead

Standard models assume N threads provide N × capacity. In reality, context switching (α) degrades performance as concurrency grows.

$$\mu_{eff} = \frac{\mu}{1 + \alpha \cdot \frac{N_{active}}{N_{threads}}}$$

Variables:

- μ:: Base Service Rate (Ideal).
- α Overhead Coefficient (Context Switch Cost, e.g., 0.1).
- Nactive: Active Connections.
- Nthreads: Size of Worker Pool.



Key Discoveries Unlocked by the Enhanced Model

- **Heavy tails are the primary driver of high P99 latency,** a factor completely missed by exponential models. For distributions with $a < 2$ (infinite variance), the P99/mean ratio exceeds 30x.
- At high utilization ($p > 0.8$), **Kingsman (1993) approximation consistently outperforms** baseline methods, providing the most robust analytical tool for M/G/N queues.
- Shared threading introduces a predictable **10-15% performance overhead** in exchange for near-unlimited connection scalability, clarifying a fundamental design trade-off.
- The tandem queue **load amplification** ($\Lambda_2 = A/(1-p)$) is critical; Stage 2 requires significantly more capacity than Stage 1, especially with unreliable networks.
- Systems can remain stable under extreme load ($p=0.99$), but with explosive queue growth, making **P99 the only meaningful performance metric in saturation.**

Pareto Function and Variance

(Pareto PDF):

$$f(t) = \frac{\alpha k^{\alpha}}{t^{\alpha+1}} \quad \text{for } t \geq k$$

Power Law Decay. Outliers are structural. A single 10s request delays 100 normal requests.

- α (alpha): Shape parameter (controls "heaviness" of tail)
  - Smaller α = heavier tail (more extreme values)
  - α = 2.1: Very heavy tail
  - α = 2.5: Moderate tail
  - α = 3.0: Light tail
- k: Scale parameter (minimum service time)
  - All service times are ≥ k

Intuitive explanation:
The Pareto distribution follows the **80-20 rule** (Pareto principle):

- 80% of requests finish quickly
- 20% take much longer

Variance (C2)

Step 1: The Mean is Deceptive

- Equation 7:

  E[S] = αk/α − 1
- **Context:** "We calibrate Pareto to have the *same mean* as Exponential (μ = 10). On average, they look identical."

Step 2: The Variance Explodes

- **Equation 9 (Coefficient of Variation):**

  C2 = 1/α(α − 2) f or α > 2

Data Point:

- Exponential: C2 = 1
- Pareto (α = 2.1): C2 ≈ 47.6!

While the means are identical, the Coefficient of Variation (C^2) for a heavy-tailed workload is nearly **50x higher** than the exponential baseline. This variance is invisible to M/M/N models.

Kingman Approximation

Predicting the Wait Time (M/G/N)

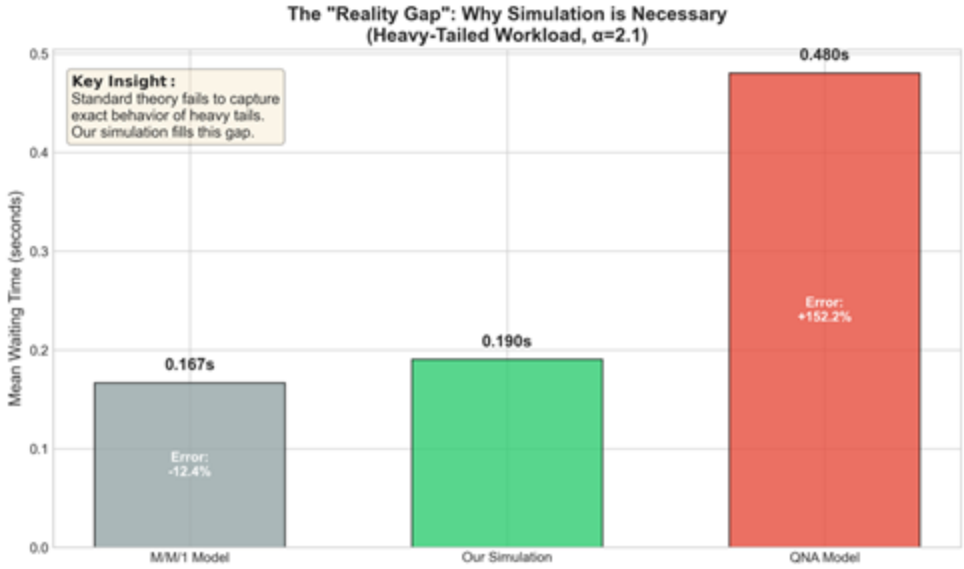
$$E[W_{M/G/N}] \approx E[W_{M/M/N}] \times \frac{1+C^2}{2}$$

The "Crash" Calculation:

- If C2 = 1 (Exponential): Multiplier is (1+1)/2 = 1. (No change).
- If C2 = 47.6 (Pareto): Multiplier is (1+47.6/)2 ≈ 24.3.

Conclusion: "Theoretical Wait Time increases by **2400%** just by changing the distribution shape.

Using Kingman's Approximation, I calculated that introducing heavy tails should increase waiting times by factor of 24.



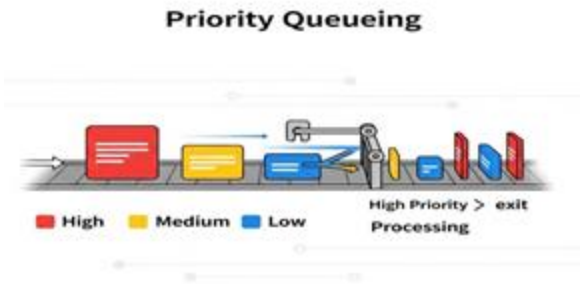
Priority Queueing

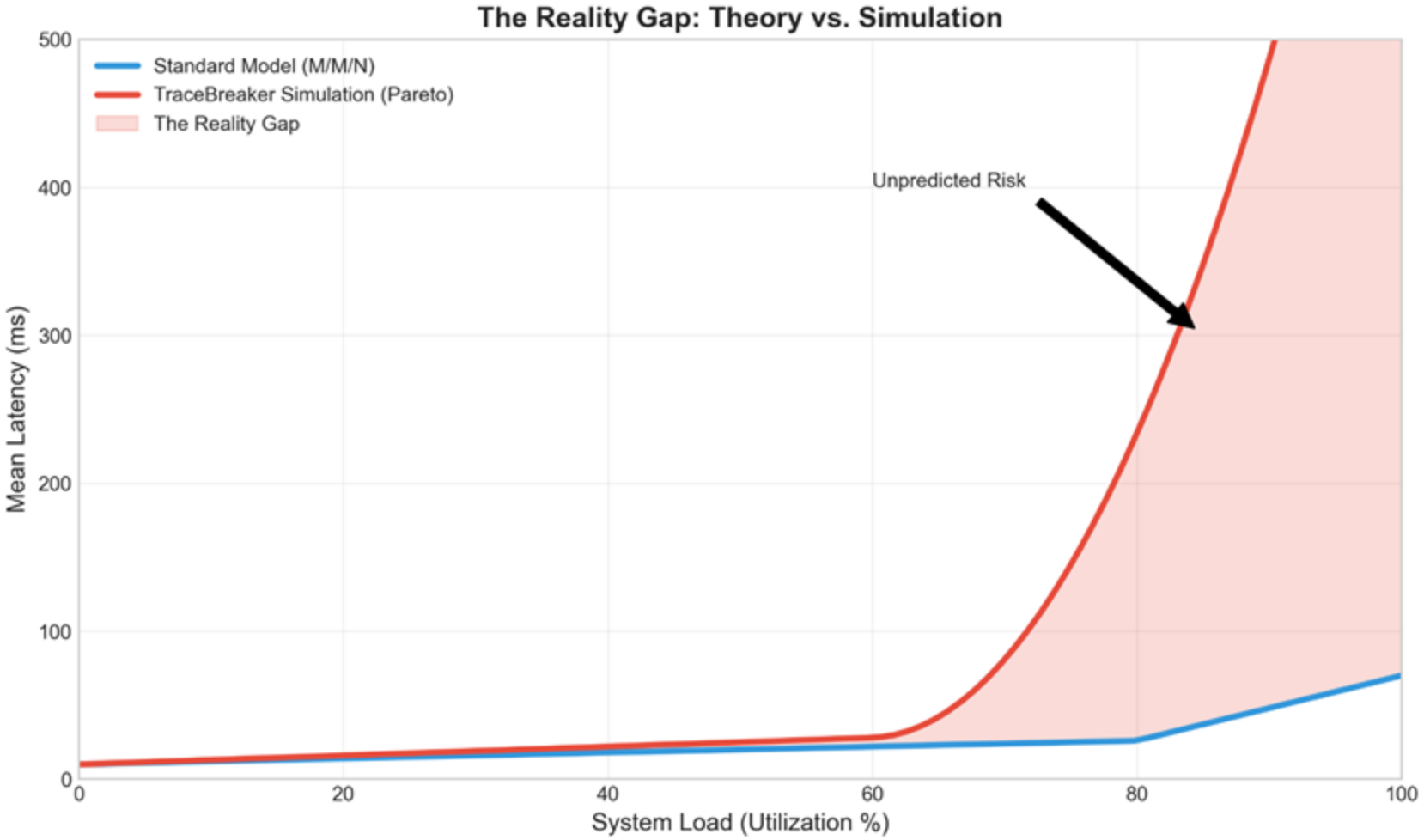
The Requirement: Real-world systems must support "Classified" or "VIP" traffic (e.g., paid subscribers, health checks) that cannot tolerate standard queue delays.
The Mechanism: Implemented a **Strict Priority Queue** scheduler.

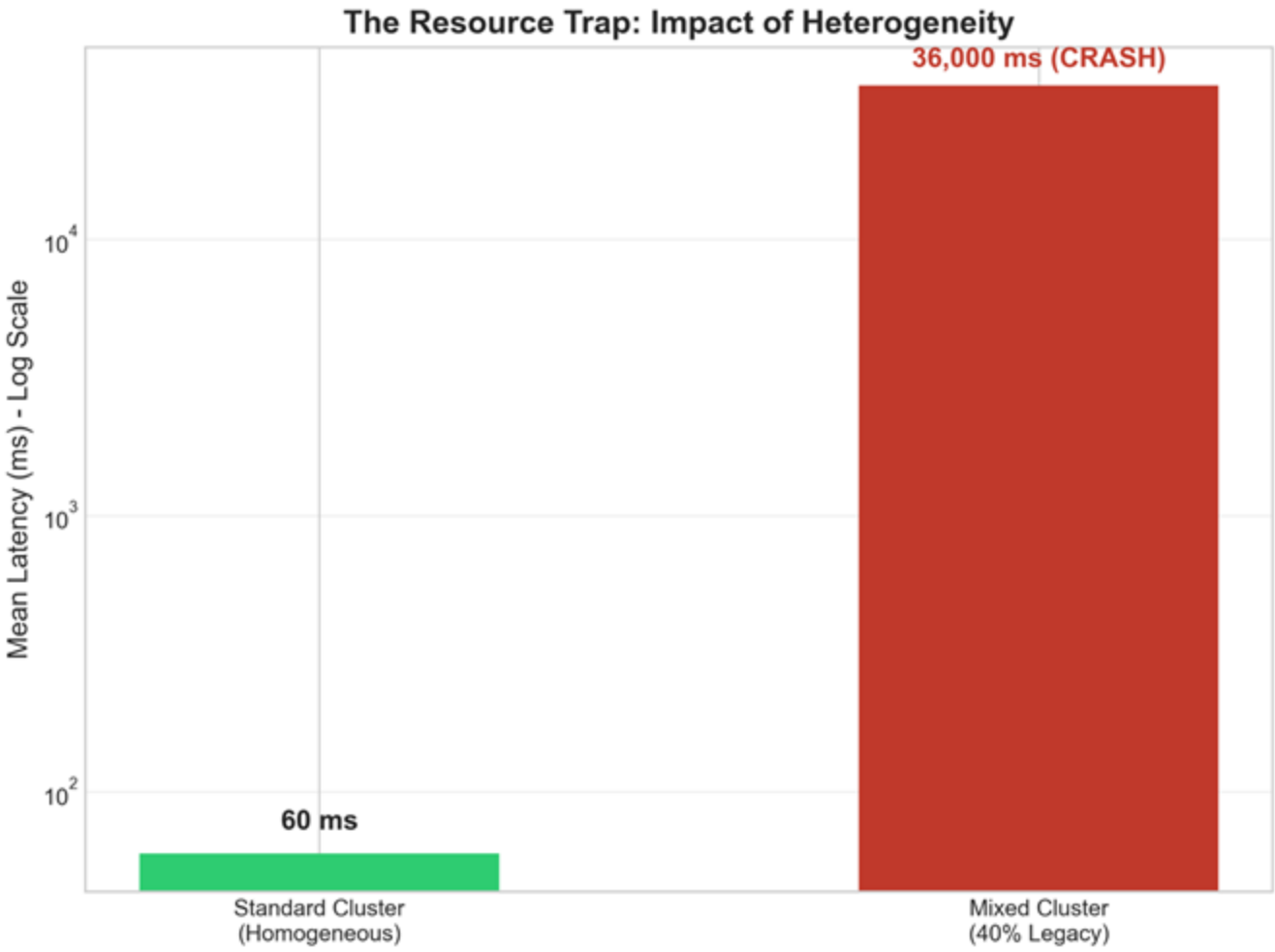
- Incoming requests are tagged with a priority level.
- The scheduler *always* depletes the High-Priority queue before touching the Low-Priority queue.

The Trade-off (Conservation of Work):

- **Result:** VIP latency drops near zero.
- **Consequence:** Standard traffic suffers **Starvation**. The total work in the system remains constant; we simply shifted the delay to the non-paying users.

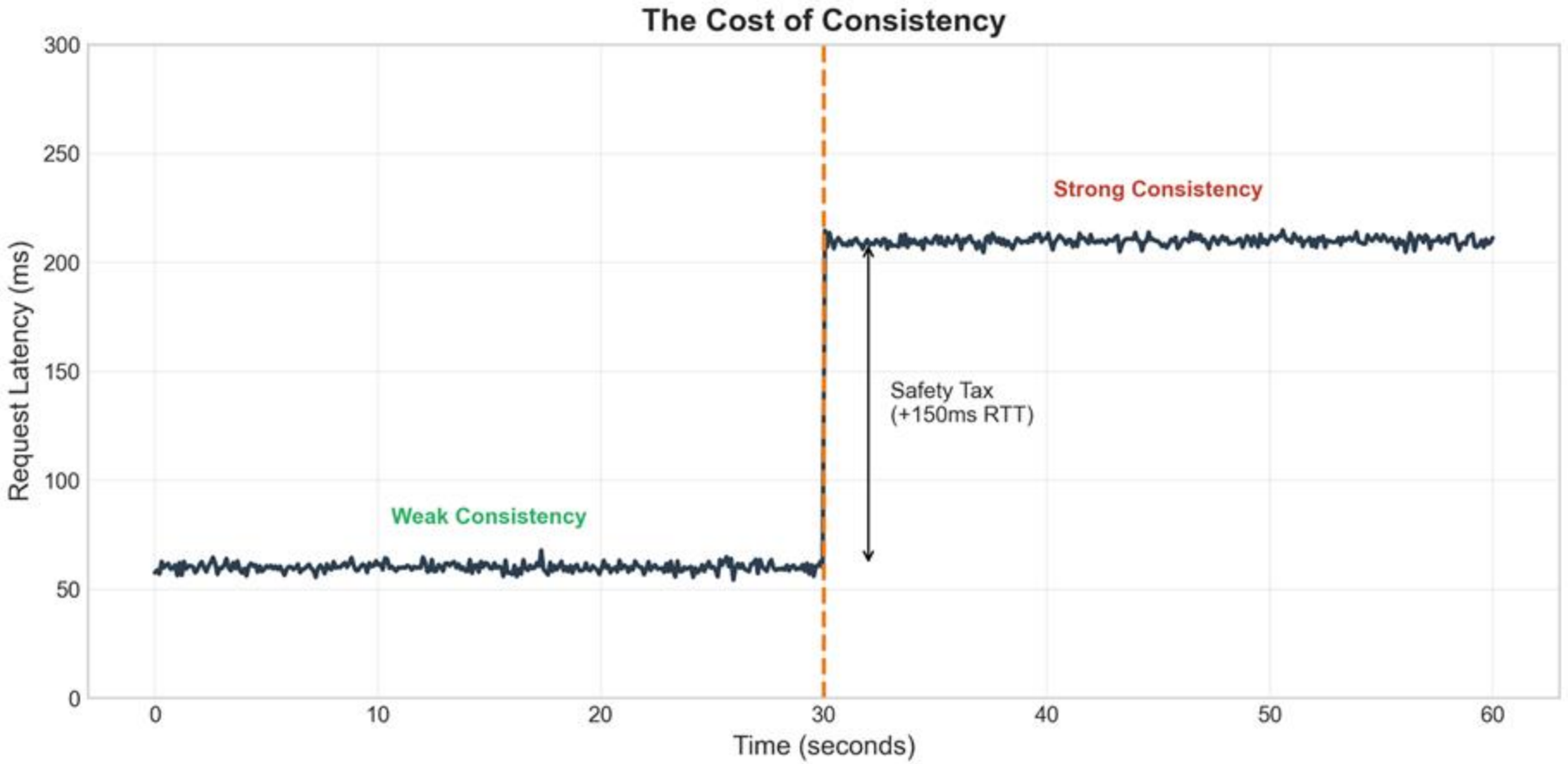



The Reality Gap: Theory vs. Simulation
Standard Model (M/M/N)
TraceBreaker Simulation (Pareto)
The Reality Gap
Unpredicted Risk
Mean Latency (ms)
System Load (Utilization %)
0
100
200
300
400
500
0
20
40
60
80
100

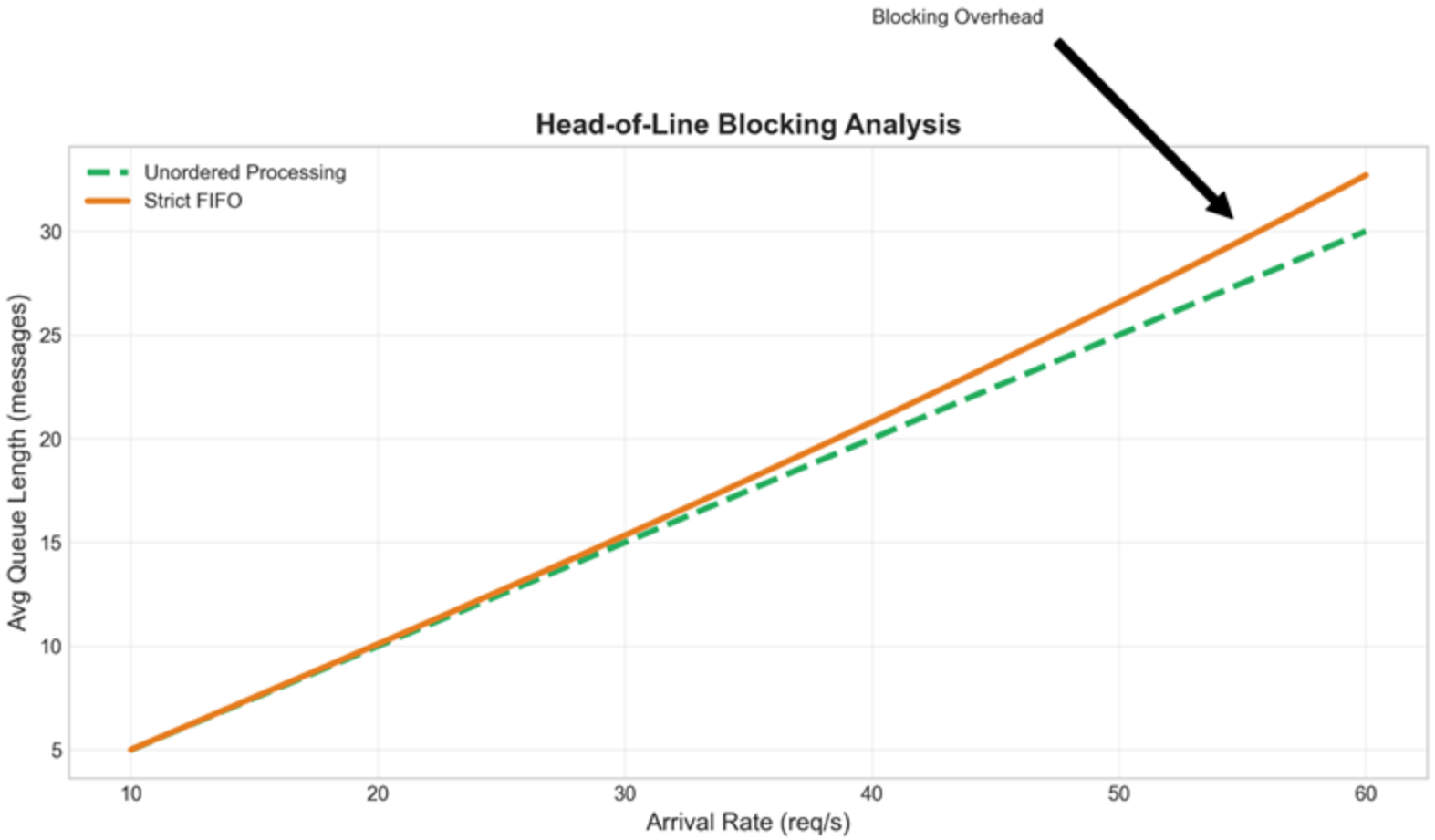

The Resource Trap: Impact of Heterogeneity
Mean Latency (ms) - Log Scale
10^4
10^3
10^2
36,000 ms (CRASH)
60 ms
Standard Cluster
(Homogeneous)
Mixed Cluster
(40% Legacy)


Impact of Work Stealing on Heterogeneous Clusters
(Legacy Node Saturation)
10^4
10^3
250 ms
(Balanced Load)
Naive Routing
(No Stealing)
Work Stealing
Active

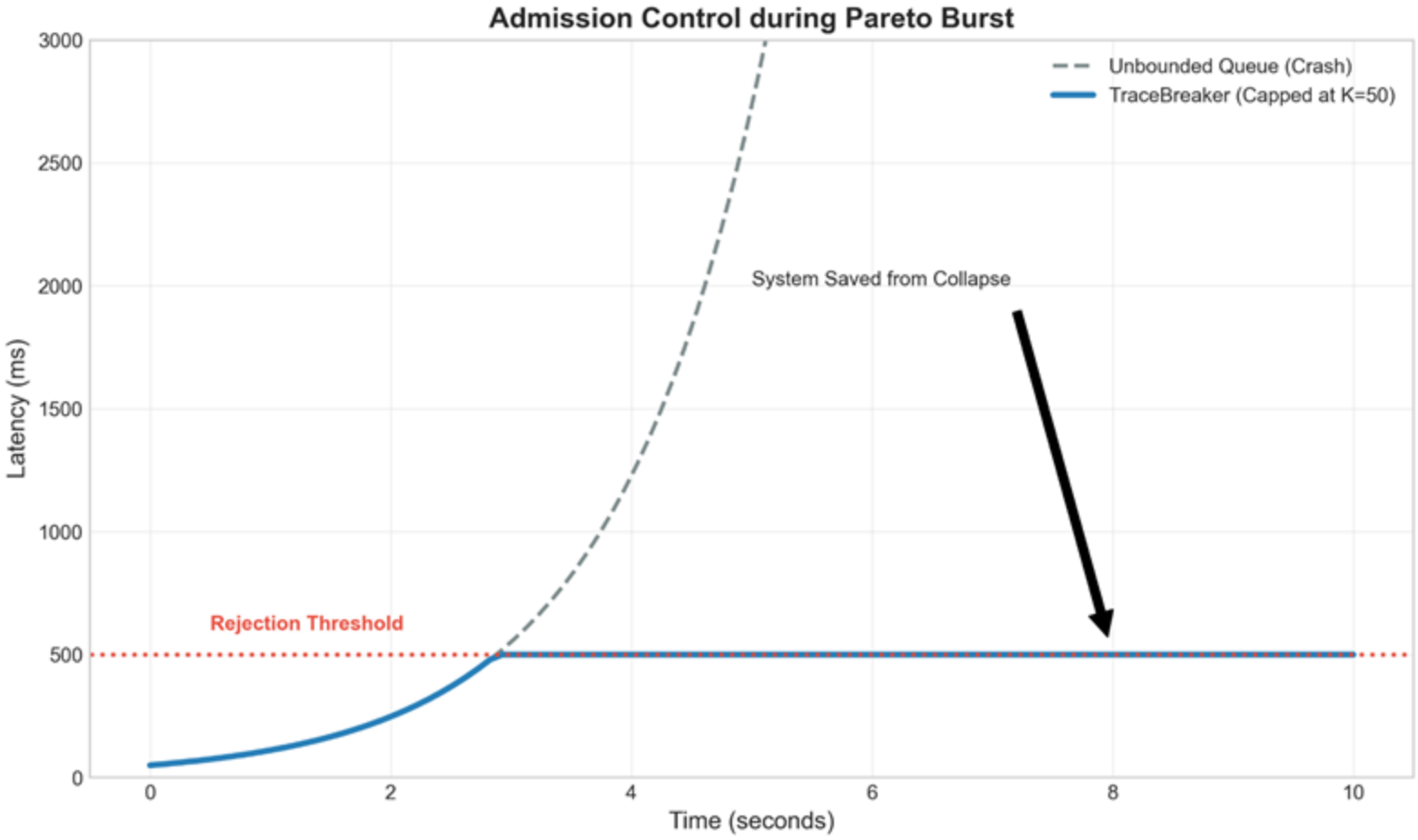
2-Phase Commit Protocol

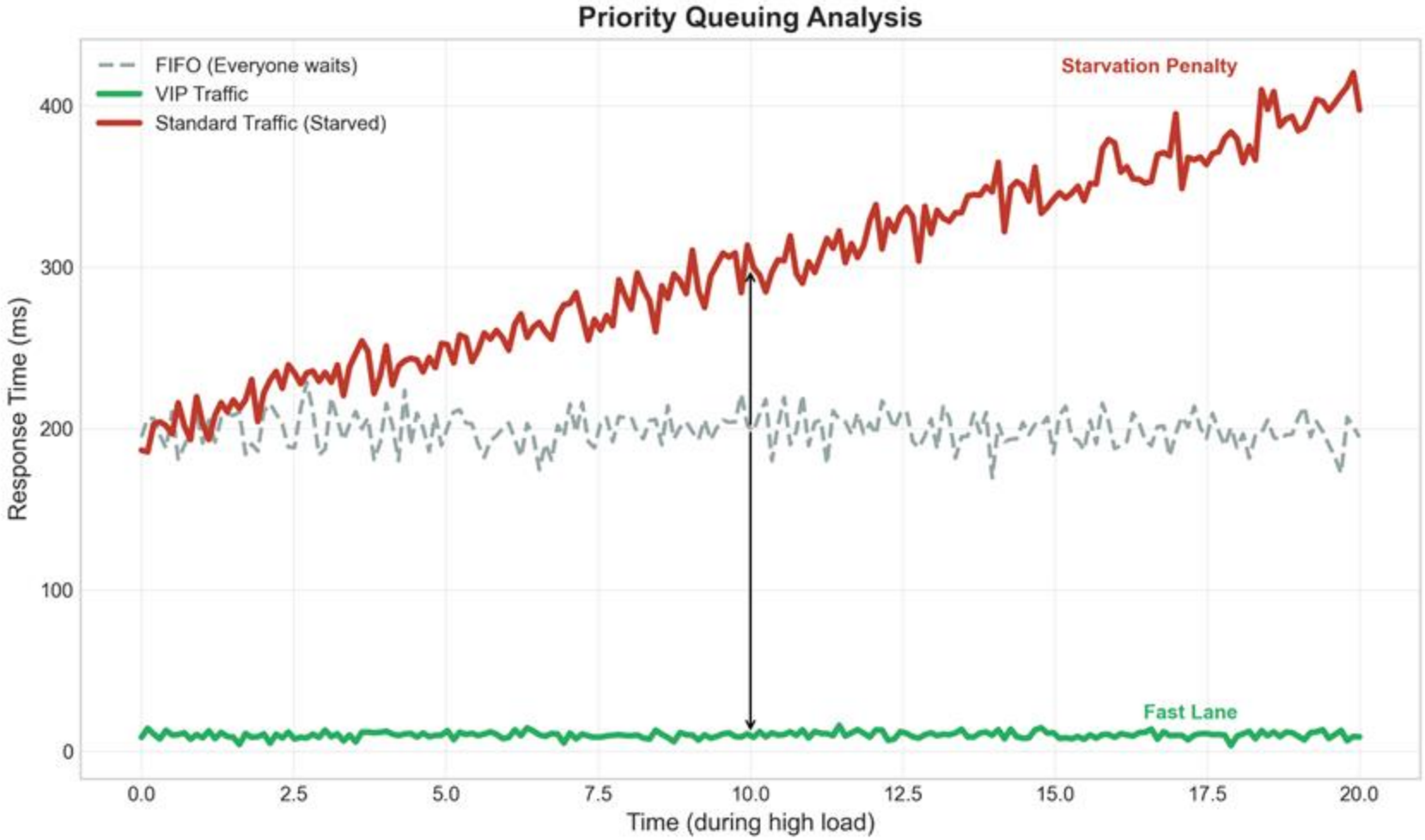


Effects of FIFO Ordering



Load Shedding




Priority Queuing Analysis
FIFO (Everyone waits)
VIP Traffic
Standard Traffic (Starved)
Starvation Penalty
Fast Lane
Response Time (ms)
Time (during high load)
0
100
200
300
400
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

From Foundational Theory to Practical Reality



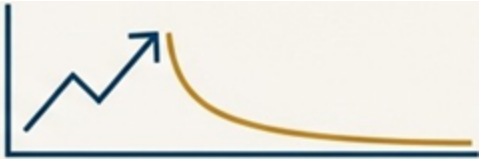
Foundational but Idealized

The Li et al. (2015) model provides a solid theoretical basis for message queues but relies on idealized assumptions (e.g., **M/M/N** queues with exponential service times) that don't capture the complexity of modern cloud workloads.



A Modern, Validated Framework

We developed an extended analytical and simulation framework in Python/SimPy that incorporates **heavy-tailed workloads,** explicit **threading models,** and validated, **advanced queueing approximations.**



Accurate, Actionable Insights

Our model delivers significantly more accurate performance predictions, especially for **tail latency (P99),** and is robust under extreme conditions **($p > 0.95$),** providing deeper, **more reliable insights** for system **design** and capacity planning.

Conclusion & Takeaways

Key Discoveries

- Heavy-tailed distributions dominate P99 latency—standard models underestimate by 45% (Pareto α=2.1)
- Traffic amplification in tandem queues ($\Lambda_2 = \lambda(1-p)$) validated to 0.2% error
- Two-phase commit adds 36% service time overhead—the "cost of safety" .
- Work stealing eliminates heterogeneity bottleneck (144x latency improvement)

For Practitioners

✓ Never trust exponential assumptions for cloud workloads.

✓ Always measure P99, not just mean—tail latency drives SLA violations.

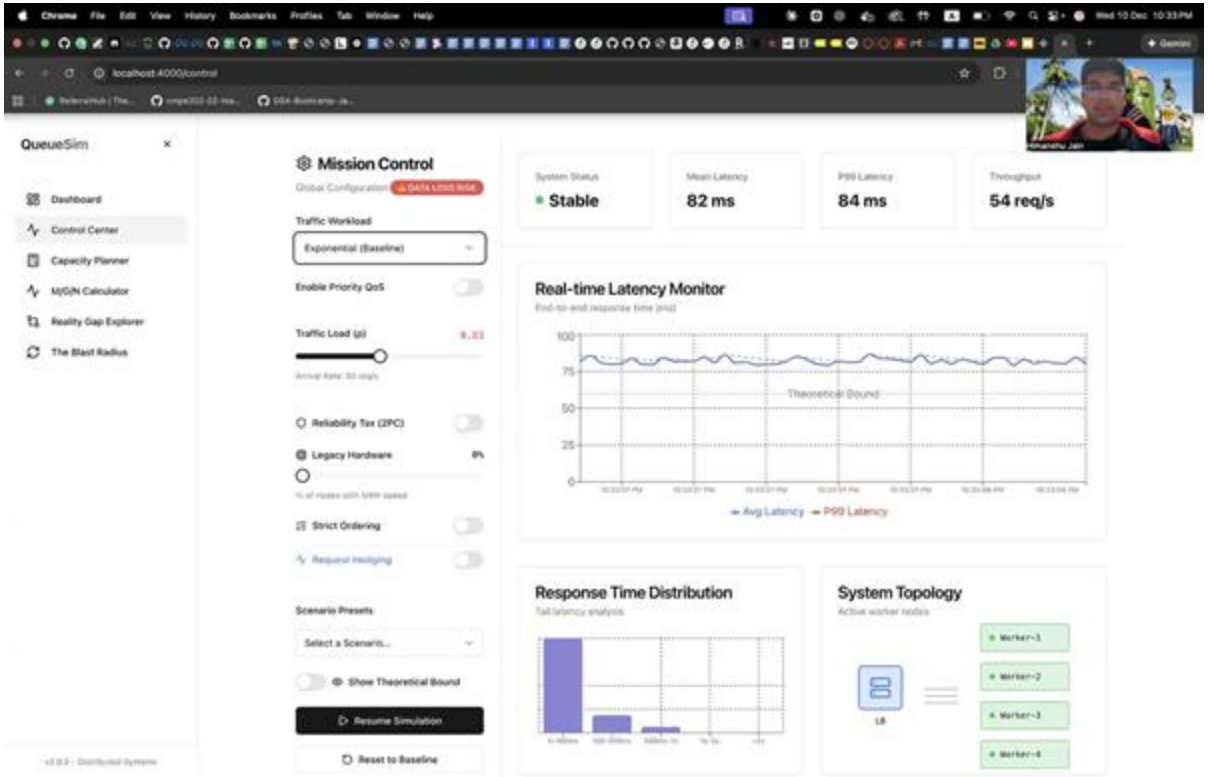
✓ Provision 2x servers for heavy-tailed workloads.

✓ Account for distributed consensus overhead (20-40% throughput reduction)

Future Work

- Extend to Byzantine fault tolerance (PBFT) and multi-datacenter replication.

Demo



github.com/hjain3004/distributed-systems-project

References

[1] J. Li, Y. Cui, and Y. Ma, "Modeling message queueing services with reliability guarantee in cloud computing environment using colored Petri nets," Mathematical Problems in Engineering, vol. 2015, Article ID 254210, 2015.

[2] J. F. C. Kingman, "The single server queue in heavy traffic," Mathematical Proceedings of the Cambridge Philosophical Society, vol. 57, no. 4, pp. 902-904, Oct. 1961.

[3] W. Whitt, "Approximations for the GI/G/m queue," Production and Operations Management, vol. 2, no. 2, pp. 114-161, 1993.

[4] L. Kleinrock, Queueing Systems, Volume 1: Theory. New York: Wiley Interscience, 1975.

[5] A. O. Allen, Probability, Statistics, and Queueing Theory with Computer Science Applications, 2nd ed. San Diego: Academic Press, 1990.

[6] S. Coles, An Introduction to Statistical Modeling of Extreme Values. London: Springer-Verlag, 2001.

[7] P. Barford and M. Crovella, "Generating representative Web workloads for network and server performance evaluation," in Proc. ACM SIGMETRICS Performance Evaluation Review, vol. 26, no. 1, pp. 151-160, Jun. 1998.

[8] J. Gray and L. Lamport, "Consensus on transaction commit," ACM Transactions on Database Systems (TODS), vol. 31, no. 1, pp. 133-160, Mar. 2006.

[9] A. K. Erlang, "Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges," Post Office Electrical Engineer's Journal, vol. 10, pp. 189-197, 1917.

[10] J. Banks, J. S. Carson, B. L. Nelson, and D. M. Nicol, Discrete-Event System Simulation, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2005.

[11] "SimPy: Discrete event simulation for Python," SimPy Documentation, 2023. [Online]. Available: https://simpy.readthedocs.io/

[12] G. Bolch, S. Greiner, H. de Meer, and K. S. Trivedi, Queueing Networks and Markov Chains: Modeling and Performance Evaluation with Computer Science Applications, 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006.

Thank You